

EF Core: Code First and Fluent API

1. Why Code First rather than Database First?

Code First	Database First
You write C# classes, and EF Core creates the database.	You start from an existing database, and EF Core generates the classes.
The classes are the single source of truth.	The database is the source of truth.
Migrations track every schema change (works well with Git).	Schema changes are harder to track.
Fits new projects and teamwork.	Fits legacy or existing databases.

Main reasons:

- You work in C#, so you do not need to design tables by hand.
- **Migrations** keep a history of database changes. Every developer can apply them with `Update-Database`.
- Version control works well, because the schema lives in code.
- It is easy to recreate the database on any machine.

Code First is not always better. If the database already exists, use Database First (`Scaffold-DbContext`).

2. Why Fluent API rather than Data Annotations?

Fluent API	Data Annotations
Configuration is in <code>OnModelCreating</code> or a configuration class.	Attributes are placed on the entity class, such as <code>[Required]</code> .
Supports all EF Core features.	Supports only some features.
Keeps entity classes clean.	Mixes database rules into entity classes.
Has the highest priority.	Is overridden by Fluent API.

Main reasons:

- **More powerful.** Some things need Fluent API, such as composite keys and complex relationships.
- **Separation of concerns.** The entity class stays a simple C# class.
- **Better organization.** With `IEntityTypeConfiguration<T>`, each entity has its own configuration file.
- **Highest priority.** If both are used, Fluent API wins.

Data Annotations are still fine for simple rules. They are shorter and are also used for validation in ASP.NET Core.

Example

```
public class BookConfigurations : IEntityTypeConfiguration<Book>
{
    public void Configure(EntityTypeBuilder<Book> builder)
    {
        builder.Property(b => b.Title)
            .IsRequired()
            .HasMaxLength(150);

        builder.Property(b => b.Price)
            .HasColumnType("decimal(8,2)");

        builder.Property(b => b.PublishedDate)
            .IsRequired(false);
    }
}
```

Short Summary

Code First gives us full control from C# code and tracks database changes using migrations. Fluent API is more powerful than Data Annotations and keeps entity classes clean.